CUDA Graphs in Pytorch Without the Pain

Daniel Galvez, AI Developer Technology Engineer

GPU Technology Conference | March 19, 2026

Agenda

What are CUDA Graphs

Benefits of CUDA Graphs

Challenges of CUDA Graphs

How torch.compile() can help

Fixing Common Errors in a Training Example

What are CUDA Graphs?

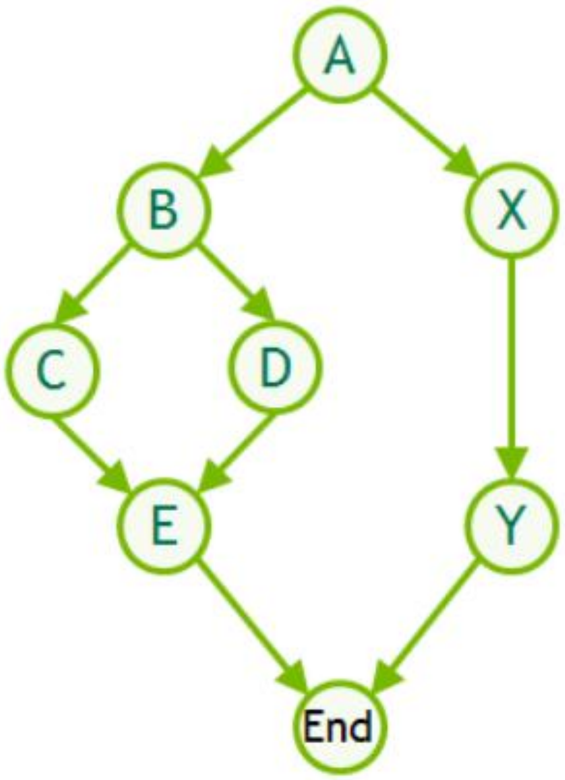
A cuda graph is a directed acyclic graph of cuda kernels, memcpies, and memsets to be executed in the order implied.

Constructed by running “CUDA Graph Capture” on an existing workload.

- cudaStreamBeginCapture() marks a cuda stream as “capturing”
- cudaStreamEndCapture() returns a “cuda graph” of work.
- No actual GPU code in the graph has been executed yet.

Executed via “graph replay” or “cudaGraphLaunch()”.

Kernel arguments used at graph capture time are reused at replay time. They cannot be changed and cannot be introspected.



Performance Benefits of Cuda Graphs

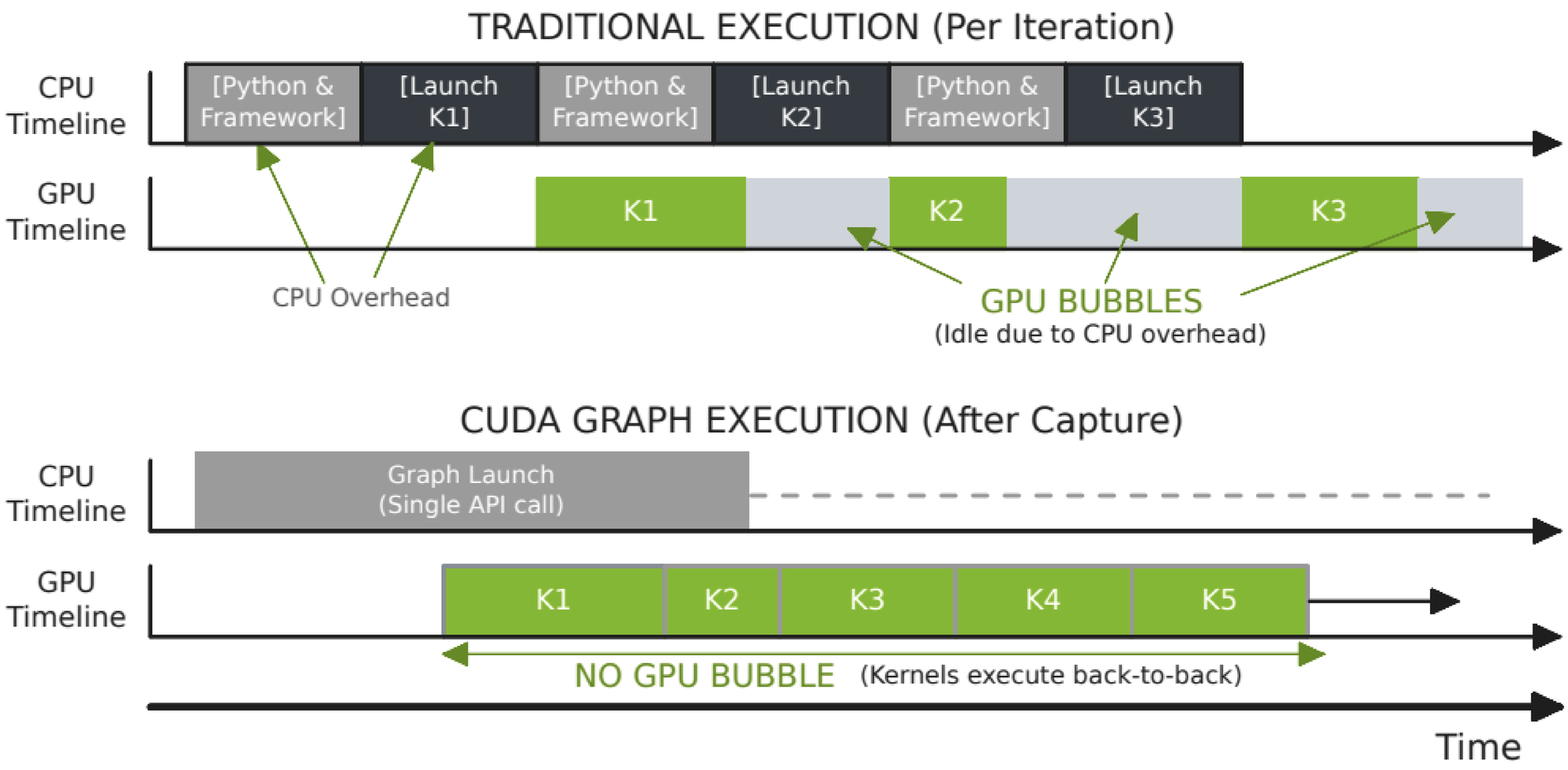
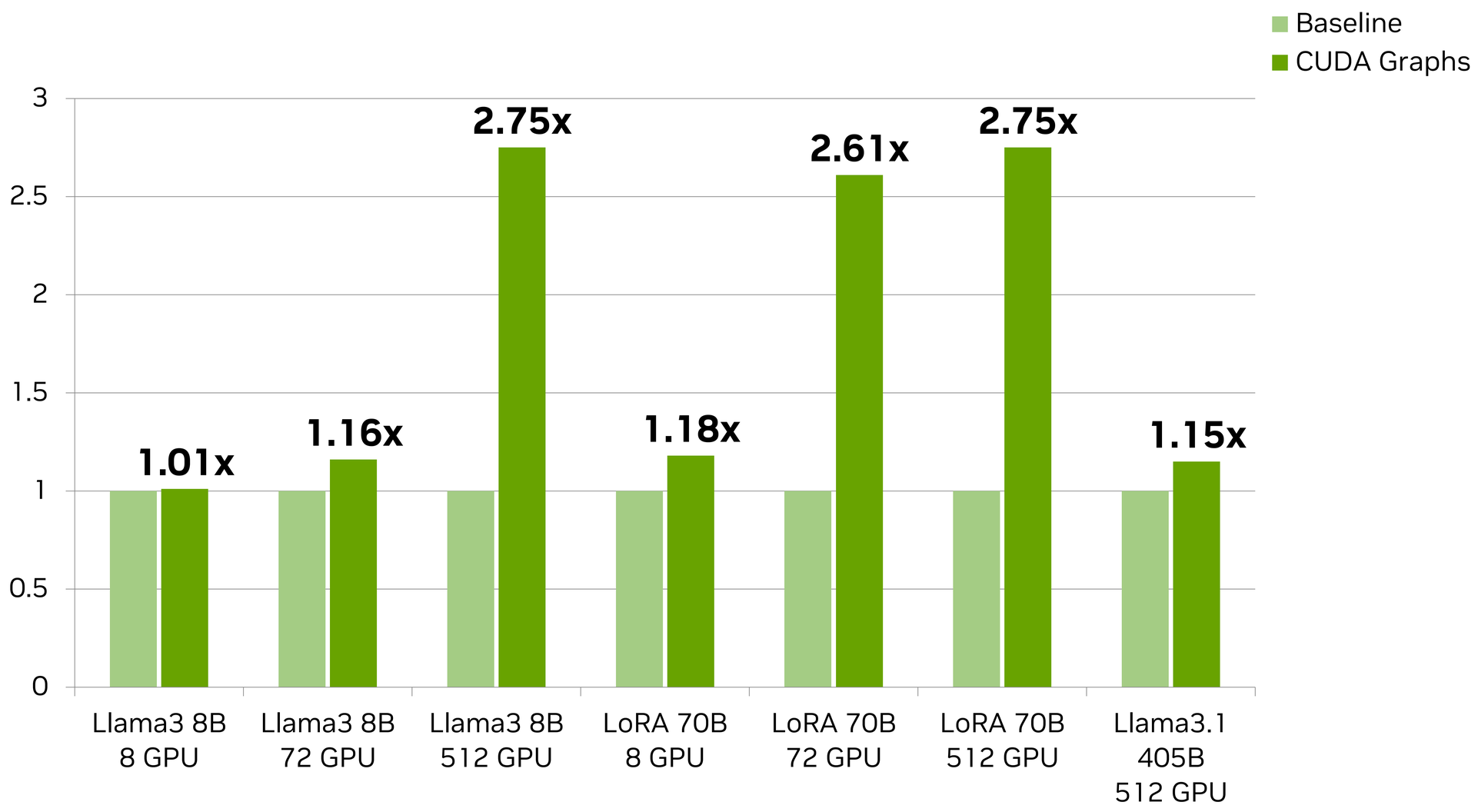


Figure 22: Traditional execution (top) versus CUDA Graph execution (bottom).

From "Scalable Training of Mixture-of-Experts Models with Megatron Core"

Performance Benefits of Cuda Graphs

MLPerf v5.1 Training



Why not always use CUDA Graphs?

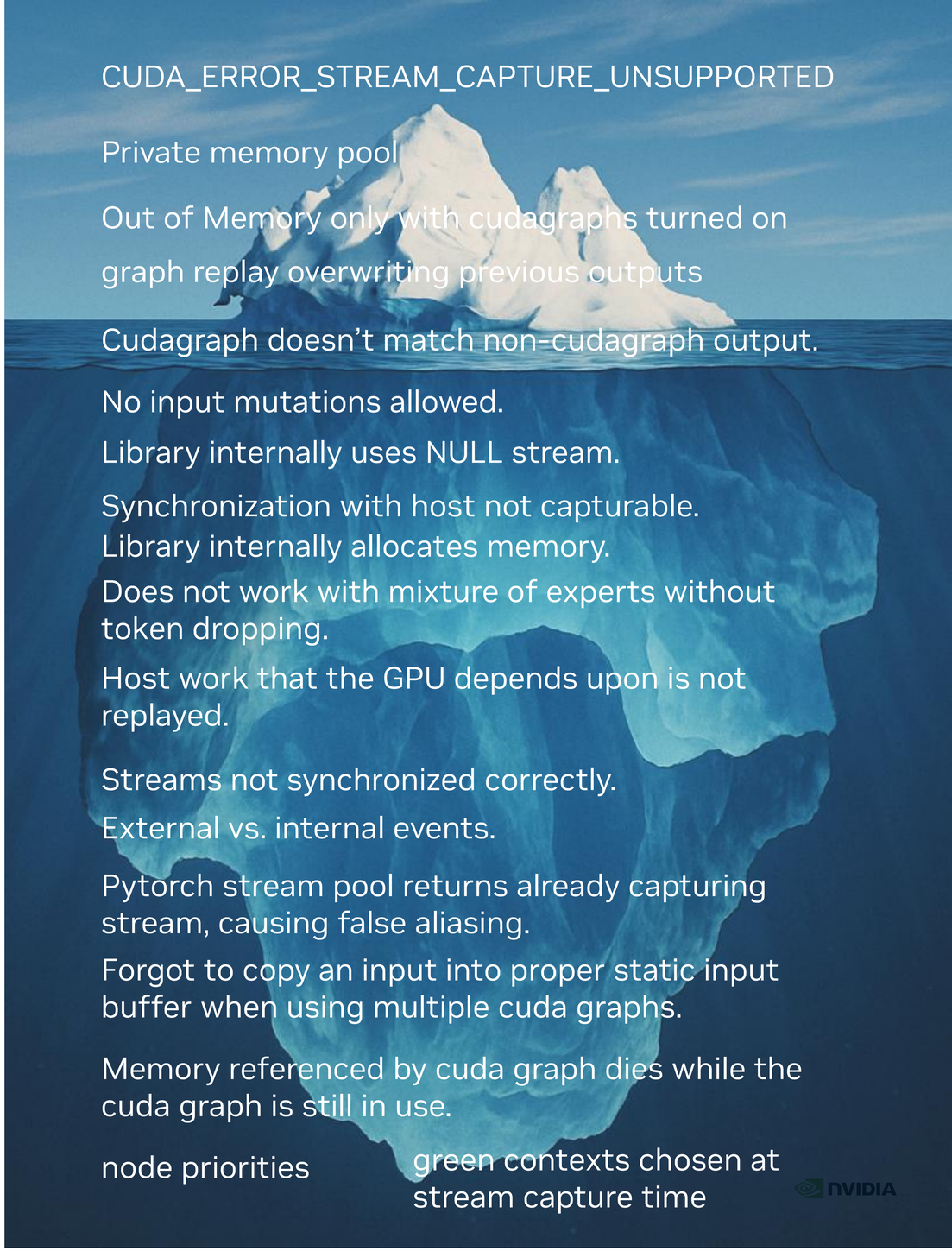
Well-formed pytorch programs may not run properly when captured as a cuda graph.

GPU code during graph capture is lazy, but CPU code during graph capture is eager.

Kernel arguments at graph capture time are reused at replay time. They cannot be changed and cannot be introspected.

1. Must copy a cuda graph's inputs into the "static input memory addresses" used at stream capture time before replay.
2. A replayed cuda graph will always write its outputs to the same memory addresses.
3. No way to introspect a cuda graph to know its input and output tensors. User responsibility to keep track.
4. Work only on fixed shape workloads

Walking through 5 common errors today.



The easiest way to use CUDA Graphs in Production

CUDA Graph Trees and Graph Partition via torch.compile()

Pros:

Just pass mode="reduce-overhead" or mode="max-autotune".

A program accepted by torch.compile() will use cuda graphs for the cuda graph compatible parts, and fall back to non cudagraph execution otherwise.

- Fxgraphs know their own inputs and outputs.
- Fxgraphs can be partitioned into cudagraph-compatible and non-cudagraph-compatible fxgraphs.

Straightforward requirement to impose on other teams.

- Successful cuda graph capture does not imply that cuda graph is correct.

Cons:

Too many graph breaks reduce benefit of cudagraphs as fxgraphs get smaller and smaller.

Many pytorch libraries introduce graph breaks or otherwise are not torch.compile() compatible.

Learn more:

CUDA Graph Trees: "CUDAGraph in a Partial Graph World" at Pytorch Conference 2024 by Elias Ellison, Meta

(FX) Graph partition: Pytorch issue #125864

Our Running Example – Training

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = 0.1

def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)
    lr = lr * 0.9
return loss
```

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Error 1: Asynchronous Data Loading Breaks Stream Capture

```
torch.AcceleratorError: CUDA error: operation
failed due to a previous error
```

- Happens during stream capture at random points, but only sometimes.

Root cause: Allocating pinned host memory in a separate thread breaks stream capture because cudaHostAlloc() is an "unsupported API"

Diagnose with:

- `export CUDA_LOG_FILE=stdout`
  - Requires CUDA 12.9 user mode driver or newer.
- `export PYTHONUNBUFFERED=1`
- `export TORCH_SHOW_CPP_STACKTRACES=1`
  - Broken on ARM; see pytorch issue #168376

```
[23:57:31.242][139912224974400][CUDA][E] API call conflicts
with a stream capture sequence initiated from another thread

[23:57:31.242][139912224974400][CUDA][E] Returning 900
(CUDA_ERROR_STREAM_CAPTURE_UNSUPPORTED) from
cuMemHostAlloc
```

Output from running with `export CUDA_LOG_FILE=stdout`

(Potentially) Unsupported APIs

CUDA_ERROR_STREAM_CAPTURE_UNSUPPORTED

By default, unsupported APIs called from any thread will break any current stream capture.

What makes a particular CUDA API unsupported during stream capture?

- Anything that synchronously creates or deletes a resource
  - cudaMalloc(), cudaFree(), cudaHostAlloc(), etc.
- Anything that makes GPU state visible to the CPU
  - cudaStreamSynchronize(), cudaEventSynchronize(), cudaEventQuery(), etc.

When should we permit an unsupported API?

- The cuda graph never references the synchronously created or deleted resource.
- OR
- The cuda graph is guaranteed to die before the synchronously created resource dies.
- OR
- The cuda graph doesn't depend upon the newly CPU-visible state.

How do we permit an unsupported API?

Library developers:

Permit via setting current thread capture mode to relaxed via cudaThreadExchangeStreamCaptureMode before calling unsupported API.

Pytorch users:

Make sure stream capture spans only one thread and is invalidated by unsupported APIs on only the current thread by passing "thread_local" to torch.cuda.graph().

Error 1: fix

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = 0.1

def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad * inv_scale)
    lr = lr * 0.9
return loss
```

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g, capture_error_mode="thread_local"):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Error 2: cuda stream capture joins with the NULL stream

UserWarning: The AccumulateGrad node's stream does not match the stream of the node that produced the incoming gradient. This may incur unnecessary synchronization and break CUDA graph capture if the AccumulateGrad node's stream is the default stream. This mismatch is caused by an AccumulateGrad node created prior to the current iteration being kept alive. …

```
File “01_vocabulary_sample_capture.py", line 30, in train
    loss.backward()
    ^^^^^^^^^^^^
torch.AcceleratorError: CUDA error: operation would make the legacy stream depend on a capturing blocking stream
```

Error 2: fix

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())

        del loss

        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g, capture_error_mode=“thread_local”):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Deleting loss will remove autograd references from warmup iteration.

Error 2: Alternative fix

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
      with torch.cuda.stream(
      ):    torch.cuda.Stream()
            loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g, capture_error_mode="thread_local"):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Use a stream other than the default (NULL)

stream for warmup.

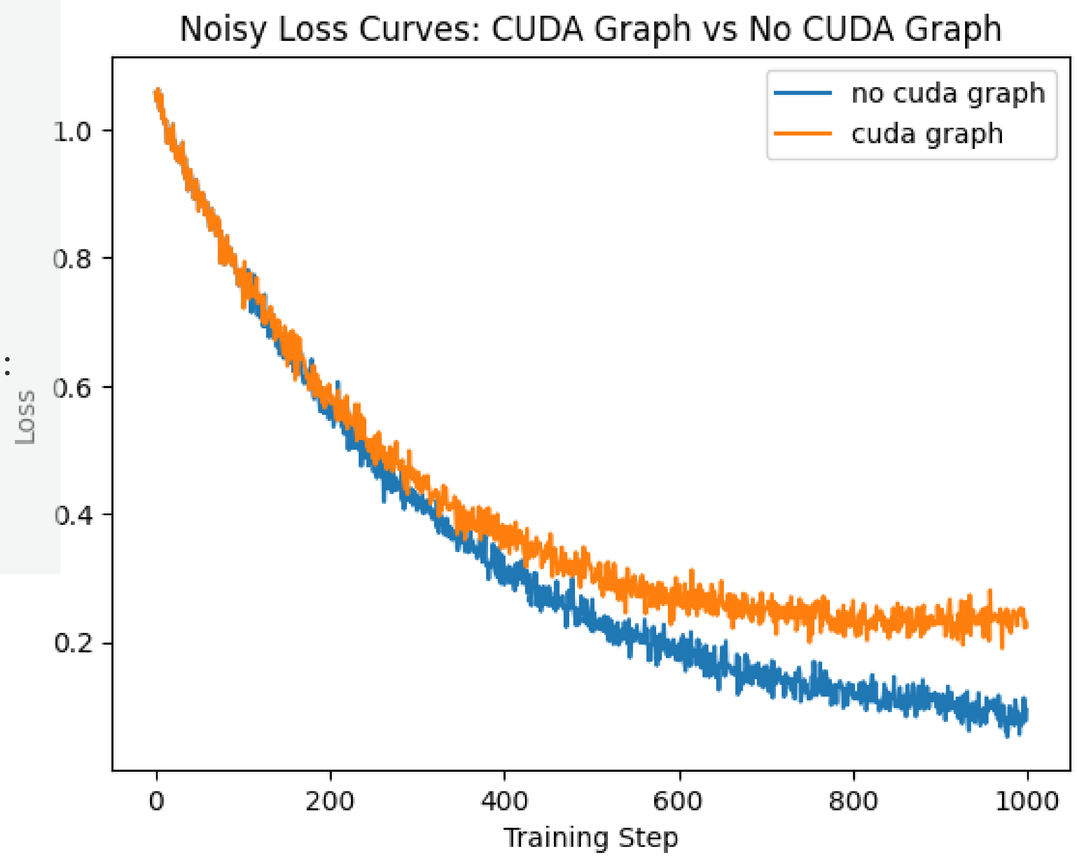
Can reduce memory fragmentation if the same stream is used for both warmup and graph capture.

- CUDA Graph Trees does this for you.

Error 3: Host Device Inconsistency

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = 0.1

def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad * inv_scale)
    lr = lr * 0.9
return loss
```



lr is a scalar python float, not a torch.Tensor.

`lr *= 0.9` is host code; it is run eagerly during stream capture.
It won't rerun during cuda graph replay.

Therefore, lr will not be updated during each replay of the cudagraph.

Error 3: initial fix

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = torch.tensor(0.1,
      device="cuda")

def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)
    lr = lr * 0.9
    return loss
```

Possible to detect this by running two stream captures with a TorchDispatchMode.

If any of the non-tensor inputs to a torch op are different, then you likely will encounter this issue.

Have not made such a debug tool yet.

See pytorch issue #167885 for updates.

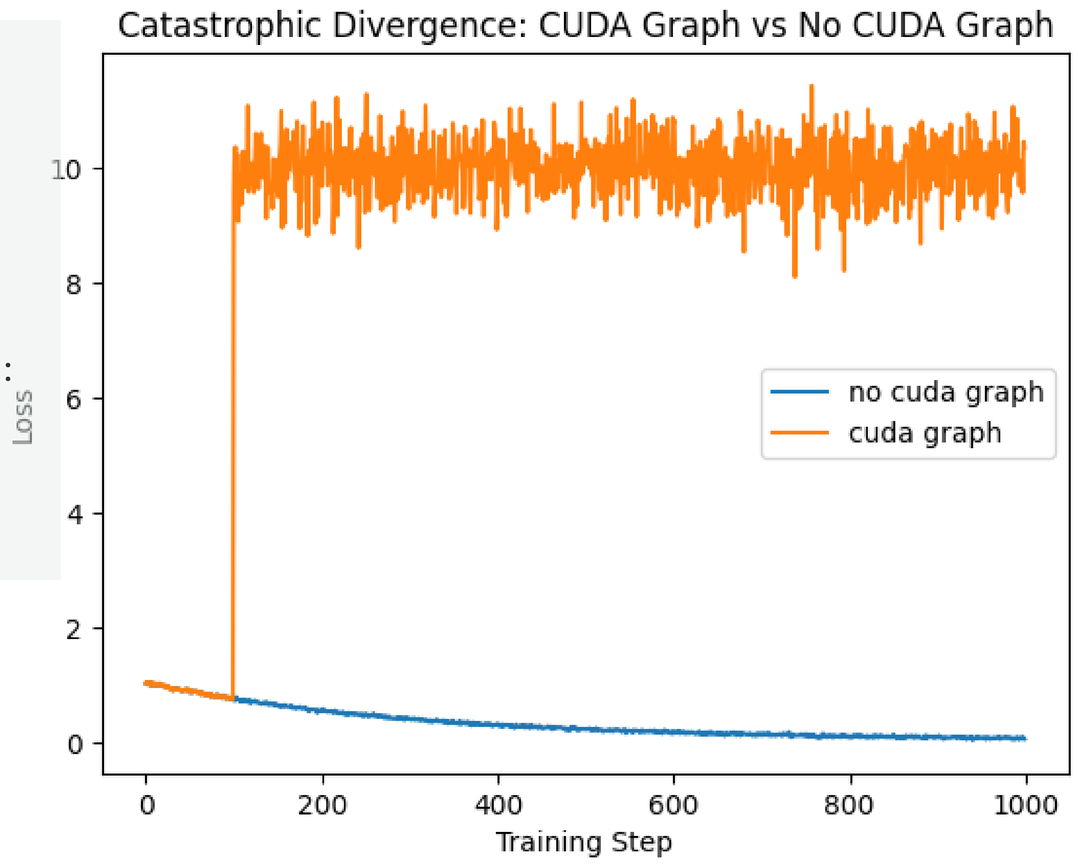
If the erroneous code were to run via torch.compile() CUDAGraph Trees, a new cudagraph would be specialized for every new value of lr.

This will be slow because cuda graphs are never reused, but a warning will trigger after enough graphs are made.

But it will never compute incorrect values.

Error 4: Use after free

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)
lr = torch.tensor(0.1, device="cuda")
def train(X, y):
    global lr
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)
    lr = lr * 0.9
    return loss
```



lr tensor is overwritten with a new lr tensor, with a different underlying memory allocation.

But the cuda graph's SGD kernels reference the old tensor.

That old tensor has no more references, so its backing memory dies and that memory can be assigned to a new tensor.

Error 4: fix

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)

lr = torch.tensor(0.1,device="cuda")

def train(X, y):
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        for param in model.parameters():
            param.sub_(lr * param.grad)

        lr.mul_(0.9)

    return loss
```

```
for i, (X, y) in enumerate(data_loader):
    if i == 0:
        loss = train(X.cuda(), y.cuda())
        print(loss.item())
        continue
    if i == 1:
        X_static = torch.empty_like(X, device="cuda")
        y_static = torch.empty_like(y, device="cuda")
        g = torch.cuda.CUDAGraph()
        with torch.cuda.graph(g,
             capture_error_mode="thread_local",
check_input_liveness=True
            ):
            loss_static = train(X_static, y_static)
    X_static.copy_(X, non_blocking=True)
    y_static.copy_(y, non_blocking=True)
    g.replay()
    print(loss_static.item())
```

Works via TorchDispatchMode. While cuda graph nodes don't know their inputs and outputs, the ops which launch these kernels do know.

See pytorch PR #174649 for details.

What if we wanted to apply a gradient NaN check?

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)

lr = torch.tensor(0.1,device="cuda")

def train(X, y):
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        all_gradients_finite = torch.all(
                                 param.grad.isfinite().all()
                                 for param in model.parameters())
        if all_gradients_finite:
            for param in model.parameters():
                param.sub_(lr * param.grad)
        lr.mul_(0.9)
    return loss
```

Error 5: Data-dependent Control Flow Breaks Stream Capture

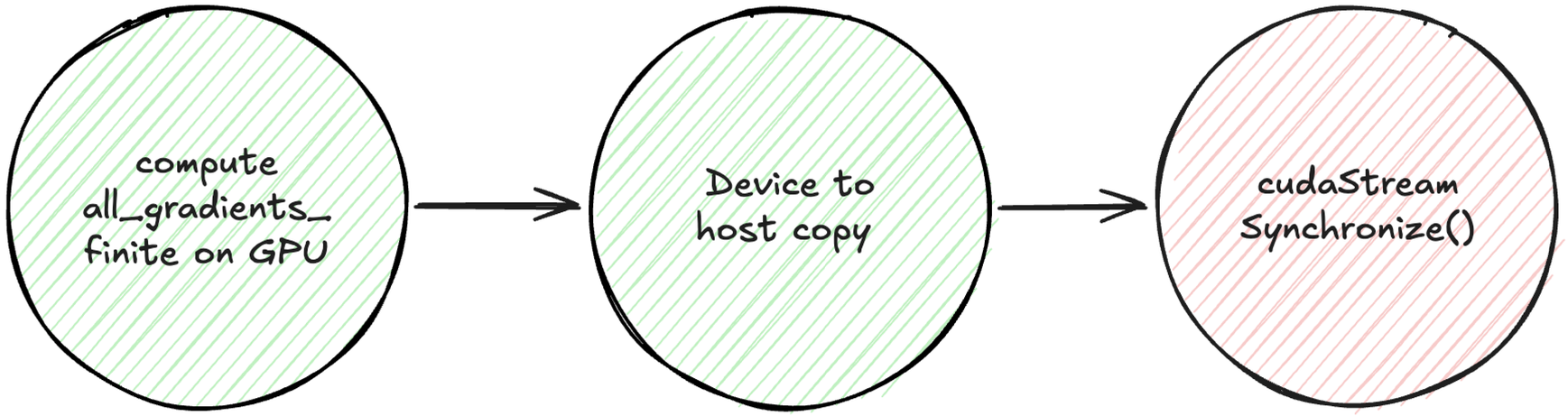
```
[12:40:35.713][140433506940736][CUDA][E] Returning 900 (CUDA_ERROR_STREAM_CAPTURE_UNSUPPORTED) from cuStreamSynchronize

 File "03_null_stream_join_autograd.py", line 40, in train

    if all_gradients_finite:

    ^^^^^^^^^^^^^^^^^^

torch.AcceleratorError: CUDA error: operation not permitted when stream is capturing
```



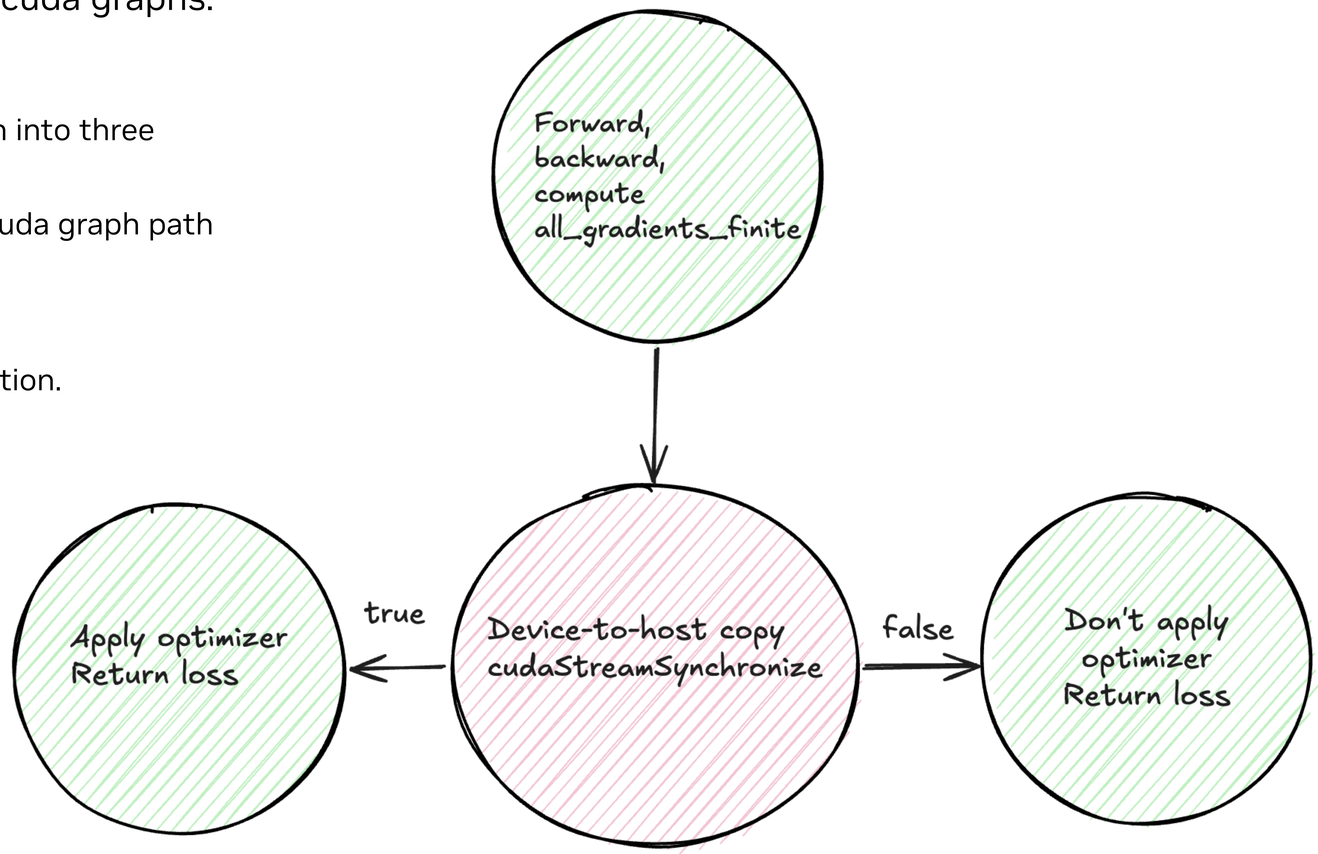
Error 5: Intrusive Fix

Partition the training workload into three cuda graphs.

Con:

- Intrusive. Need to split previous train() function into three functions
- Need to duplicate control flow logic between cuda graph path and non cuda graph path.

torch.compile() will do this for you via graph partition.



Error 5: Non-Intrusive Fix

CUDA Graph Conditional Node via torch.cond()

```
model = torch.nn.Linear(16, 4).cuda()
criterion = torch.nn.MSELoss()
data_loader = DataLoader(RandomDataset(), batch_size=32,
                         shuffle=False,
                         pin_memory=True, num_workers=1)

lr = torch.tensor(0.1,device="cuda")

def train(X, y):
    model.zero_grad(set_to_none=True)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    with torch.autograd.grad_mode.set_multithreading_enabled(False):
        loss.backward()
    with torch.no_grad():
        all_gradients_finite = torch.all(param.grad.isfinite().all() for param in model.parameters())
        def true_fn():
            for param in model.parameters():
                param.sub_(lr * param.grad)
        def false_fn():
            pass
        torch.cond(all_gradients_finite,true_fn,false_fn)
    lr.mul_(0.9)
    return loss
```

Error 5: Non-Intrusive Fix

Pros:

Non-intrusive, have just one cuda graph.

No repeated logic.

Cons:

Introduced into pytorch in February 2026.

Will be in pytorch 2.12.

Works only for if-else statements.

torch.while_loop() and torch.switch() cuda graph support on the way.

- Pytorch issue #175001
- Pytorch issue #175891

Conclusion

TL;DR

torch.compile() is the best way to enable cuda graphs incrementally.

We are constantly making more operations cudagraph compatible.

We are constantly making more tools for diagnosing errors.

- Follow pytorch issue #167885

Questions:

- Pytorch Github issues. Tag @galv (me).
- Email: dgalvez@nvidia.com
- x.com/memorypaladin

Many more errors not covered today.
Learn more at
https://docs.nvidia.com/dl-cuda-graph/latest/

NVIDIA